

Encounter Design Crew

Six agents that design a floor of **Kindled** against its own measured frame budget

Aaron Low · 27 July 2026 · Python 3.8+, standard library only

Code: `crew/kindled_crew.py` · ReadMe: `crew/README.md` · Output: `crew/out/`

Run it: `py crew/kindled_crew.py`

1 The game, and what the crew makes for it

Kindled is a single-player top-down roguelike in Unreal Engine 5.8. You carry the only fire in a pitch-dark world, and an army gathers in your light because outside it they die. You command that army — three to ten units early, hundreds by late run — through four broad stances rather than unit selection. It is the capstone game for this course; the design document is Assignment #2's final GDD, and the playable prototype lives in the same repository as this crew.



The game this crew designs for. The bright core is the bearer's flame; the dithered falloff is the lit floor. 120 retinue hold formation inside it while enemies approach from the dark at the top of frame — that is 370 entities in one shot. **Every body on this screen costs draw time**, which is the constraint the crew exists to enforce.

The game's defining constraint is **entity count against frame budget**. Every enemy on screen costs draw time, and this project has *measured* that cost (`docs/perf/BUDGETS.md`). So encounter design in Kindled is not a taste question — it is a budget question with a hard ceiling at 16.6 ms per frame.

This crew turns the project's measured canon into a **DataTable-ready encounter specification**: the wave composition for one floor of the vertical slice, priced against the real frame budget and checked against the locked four-value palette. One run writes three files:

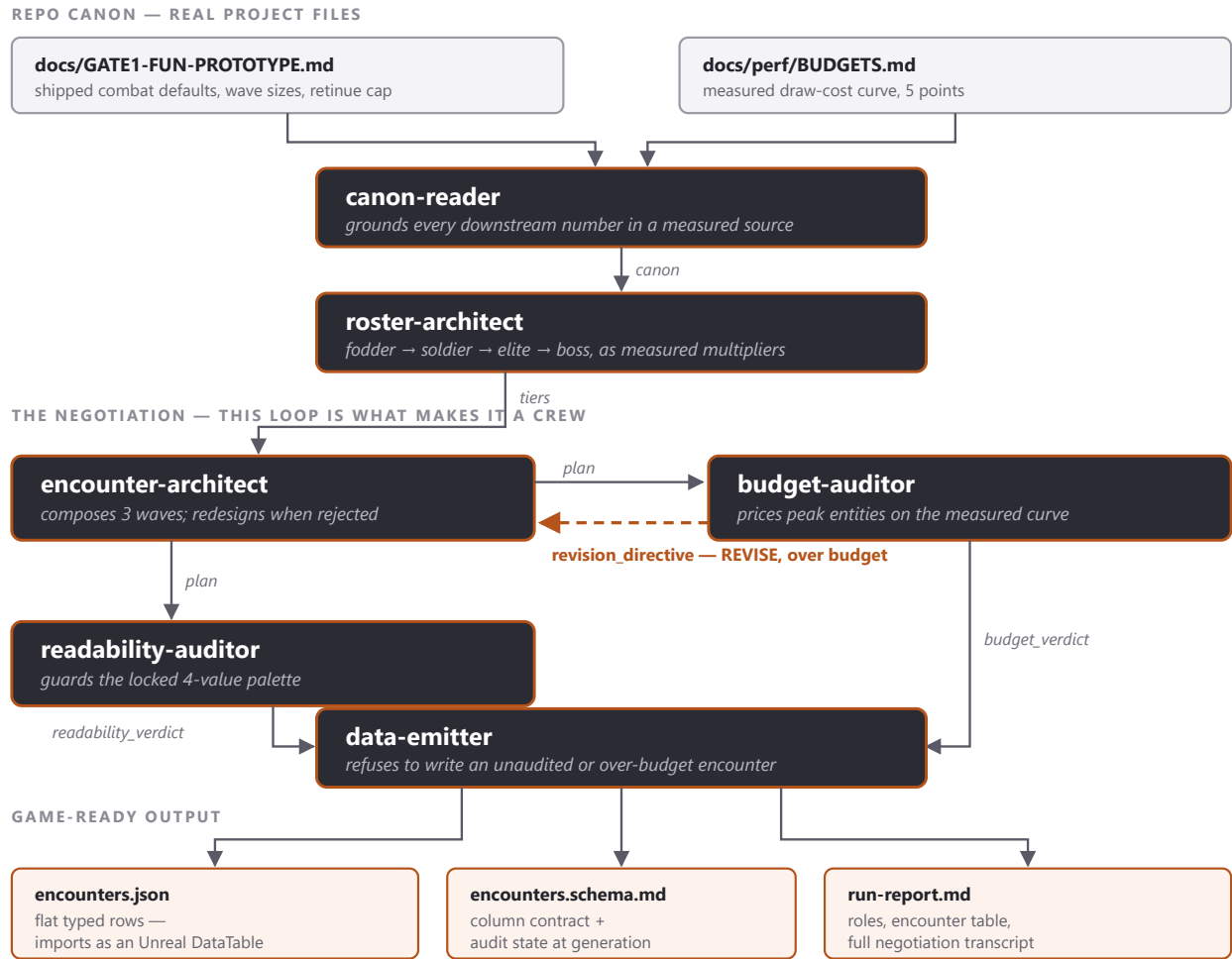
ARTIFACT	WHAT IT IS
<code>out/encounters.json</code>	Flat, typed rows that import directly as an Unreal DataTable — the game data the encounter actually ships as
<code>out/encounters.schema.md</code>	The column contract for that JSON, plus both audit verdicts at generation time
<code>out/run-report.md</code>	Every agent's declared role, the encounter table, the budget verdict, and the full negotiation transcript

This is game-ready output rather than a document *about* game-ready output: `encounters.json` has a unique `Name` row key, scalar columns only and no nesting — exactly what Unreal's DataTable importer requires, and the same contract the project's existing data files follow.

WHY RAW ORCHESTRATION AND NOT CREWAI

No dependencies: Python 3.8+ standard library, no network calls, no API key, nothing to install. That is deliberate. The rubric grades whether the crew *runs*, and a crew that needs a live API key is a crew that can fail at grading time for reasons that have nothing to do with its design. The assignment permits "CrewAI **or raw orchestration code**" — this is the latter, taken seriously: enforced I/O contracts, a shared blackboard, and a real negotiation loop.

2 Architecture



Every agent declares the blackboard keys it may read and write. The Blackboard enforces those declarations at runtime, so removing any agent halts the run at the first consumer that needs its artifact — the graph above is executable, not drawn.

The crew. Solid arrows are artifacts moving forward. The dashed ember arrow is the one that matters: **budget-auditor can send the plan back**, and the crew loops until it converges. That feedback edge is the difference between a pipeline and a crew.

Mermaid source

```
flowchart TD
    subgraph SOURCES[Repo canon - real project files]
        G1["docs/GATE1-FUN-PROTOTYPE.md<br/>shipped combat defaults, wave sizes"]
        BU["docs/perf/BUDGETS.md<br/>measured draw-cost curve"]
    end

    A1["canon-reader<br/><i>grounds the run</i>"]
    A2["roster-architect<br/><i>derives the tier ladder</i>"]
    A3["encounter-architect<br/><i>composes the waves</i>"]
    A4["budget-auditor<br/><i>prices it, can reject</i>"]
    A5["readability-auditor<br/><i>guards the 4-value palette</i>"]
    A6["data-emitter<br/><i>writes game data</i>"]

    OUT1["out/encounters.json<br/>UE DataTable rows"]
    OUT2["out/encounters.schema.md"]
    OUT3["out/run-report.md"]

    G1 --> A1
    BU --> A1

    A1 -->|canon| A2
    A1 -->|canon| A3
    A1 -->|canon| A4
    A1 -->|canon| A5
    A1 -->|canon| A6

    A2 -->|tiers| A3
    A2 -->|tiers| A6

    A3 -->|plan| A4
    A4 -. ->|"revision_directive: REVISE, over budget"| A3
    A4 -->|budget_verdict| A6

    A3 -->|plan| A5
    A5 -->|readability_verdict| A6

    A6 --> OUT1
    A6 --> OUT2
    A6 --> OUT3

    classDef agent fill:#2b2b33,stroke:#b4531a,stroke-width:2px,color:#ffffff
    classDef src fill:#f6f6f7,stroke:#9a9aa4,color:#222222
    classDef out fill:#fdf3ec,stroke:#b4531a,color:#222222
    class A1,A2,A3,A4,A5,A6 agent
    class G1,BU src
    class OUT1,OUT2,OUT3 out
```

3 The agents — role, input, output

Each agent declares the blackboard keys it may read and write. The `Blackboard` class **enforces those declarations at runtime** — an agent that touches an undeclared key raises `ContractViolation`. That is why the graph in §2 is executable rather than drawn.

AGENT	INPUT	OUTPUT	WHAT IT DOES
canon-reader	repo markdown	canon	Parses <code>GATE1-FUN-PROTOTYPE.md</code> and <code>BUDGETS.md</code> for the shipped wave sizes, the retinue baseline and the measured draw-cost curve. Nothing downstream is permitted to invent a number.
roster-architect	canon	tiers	Derives the fodder → soldier → elite → boss ladder as <i>multipliers over the measured baseline</i> , and prices each tier in encounter-budget points.
encounter-architect	canon, tiers, revision_directive	plan	Composes the three waves — how many of which tier, spawn mode, breather length — and redesigns when an auditor rejects .
budget-auditor	canon, plan	budget_verdict, revision_directive	Projects peak concurrent entities onto the measured cost curve by interpolation. Returns PASS, or REVISE with a directive naming exactly how many bodies are affordable.
readability-auditor	canon, plan	readability_verdict	Checks each wave's distinct enemy types against the free palette values, so the player can still parse the fight. Flags waves that must be carried by silhouette rather than value.
data-emitter	canon, tiers, plan, both verdicts	the three files	Flattens the approved plan into <code>DataTable</code> rows, writes the schema and the run report, and refuses to emit an unaudited or over-budget encounter .

No agent can be removed

The rubric asks whether any agent could be dropped without breaking the pipeline. This crew answers with a flag rather than a promise — `--drop <agent>` runs the crew without one and lets the contract system stop it:

```
$ py crew/kindled_crew.py --drop budget-auditor

!! RUNNING WITHOUT 'budget-auditor' - demonstrating that the pipeline depends on it

[canon-reader]
- read 2 canon source(s): docs/GATE1-FUN-PROTOTYPE.md, docs/perf/BUDGETS.md
[roster-architect]
- derived 4 tiers over baseline 130HP/30DPS
--- negotiation round 1 ---
[encounter-architect]
- composed W1: 250 bodies | W2: 450 bodies | W3: 701 bodies
[readability-auditor]
- CROWDED - max 4 distinct enemy types against 2 free palette values
[data-emitter]

PIPELINE BROKEN - data-emitter needs 'budget_verdict', but no agent has produced it.
An upstream agent is missing from the crew - the pipeline cannot proceed.

exit code 1
```

Every agent fails this way, at a different point, for a different missing artifact. Drop `roster-architect` and `encounter-architect` halts on a missing `tiers`; drop `canon-reader` and nothing starts at all.

4 It runs — a real floor-1 run, verbatim

```
$ py crew/kindled_crew.py

=====
KINDLED - ENCOUNTER DESIGN CREW
raw orchestration | stdlib only | 6 agents | floor 1
=====

[canon-reader]
- read 2 canon source(s): docs/GATE1-FUN-PROTOTYPE.md, docs/perf/BUDGETS.md
- waves=[250, 450, 700] retinue_cap=120 perf curve has 5 measured points

[roster-architect]
- derived 4 tiers over baseline 130HP/30DPS: fodder(58HP), soldier(130HP),
  elite(416HP), boss(2340HP)

--- negotiation round 1 ---
[encounter-architect]
- composed W1: 250 bodies (250pts) | W2: 450 bodies (674pts) | W3: 701 bodies (1698pts)
[budget-auditor]
- PASS - peak 821 entities ~ 11.12ms (5.48ms headroom)

[readability-auditor]
- CROWDED - max 4 distinct enemy types in a wave against 2 free palette values

[data-emitter]
- wrote 7 DataTable rows -> crew/out/encounters.json
- wrote column contract -> crew/out/encounters.schema.md

=====
DONE - 7 encounter rows, 1 negotiation round(s)
=====
```

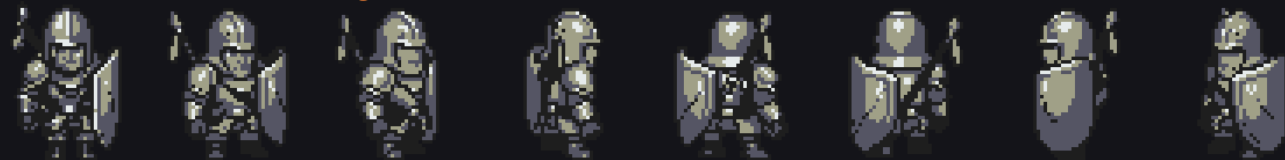
THE UNITS — T_Swarm_2bit, the shipping atlas

8 rotations, 4 palette values, no anti-aliasing

BROOD — what the dark sends



RETINUE — liberated soldiers, the Vanguard line



What the rows become. The shipping sprite atlas the encounter data drives. `roster-architect`'s *fodder* and *soldier* tiers are the top row; the retinue the player commands is the bottom. The `readability-auditor` exists because of this image: at four palette values, once the lit floor and the retinue have taken theirs, only two are left to keep enemy types apart.

What it wrote

```
{
  "_meta": {
    "game": "Kindled", "floor": 1, "generated": "2026-07-27",
    "canon_sources": ["docs/GATE1-FUN-PROTOTYPE.md", "docs/perf/BUDGETS.md"],
    "grounded_in_repo_docs": true,
    "budget_verdict": { "peak_concurrent_entities": 821, "projected_frame_ms": 11.12,
                       "frame_budget_ms": 16.6, "headroom_ms": 5.48, "status": "PASS" }
  },
  "rows": [
    { "Name": "W1_fodder", "Wave": 1, "Tier": "fodder", "Count": 250, "HP": 58,
      "DPS": 18, "BudgetCost": 1, "SpawnMode": "ring", "BreatherAfterSeconds": 6 },
    { "Name": "W3_elite", "Wave": 3, "Tier": "elite", "Count": 42, "HP": 416,
      "DPS": 54, "BudgetCost": 12, "SpawnMode": "arena_entrances", "BreatherAfterSeconds": 0 },
    { "Name": "W3_boss", "Wave": 3, "Tier": "boss", "Count": 1, "HP": 2340,
      "DPS": 78, "BudgetCost": 60, "SpawnMode": "arena_entrances", "BreatherAfterSeconds": 0 }
  ]
}
```

5 What a real run found

The crew is not a demonstration that prints a pre-written answer. It does arithmetic against measured data and reaches conclusions the author did not hand it.

Floor 1 passes comfortably. The shipped Gate-1 wave structure — 250 / 450 / 700 brood plus a 120-unit retinue cap — peaks at **821 concurrent entities $\approx$ 11.12 ms**, leaving 5.48 ms of headroom.

Floor 3 does not. At the escalated density the vertical slice calls for, it peaks at **1,871 entities $\approx$ 37.29 ms — 2.3 $\times$ over budget**. The budget-auditor rejects it, tells the architect only 956 bodies are affordable, and the two converge over three rounds:

```
$ py crew/kindled_crew.py --floor 3

--- negotiation round 1 ---
[encounter-architect] composed W3: 1751 bodies (4155pts)
[budget-auditor]       REWISE - peak 1871 entities ~ 37.29ms, over budget by 20.69ms;
                        only 956 bodies are affordable, sending back

--- negotiation round 2 ---
[encounter-architect] revising; re-scaling to x1.36 of the shipped wave sizes
[budget-auditor]       REWISE - peak 1077 entities ~ 16.62ms, over budget by 0.02ms
--- negotiation round 3 ---
[encounter-architect] composed W3: 956 bodies (2292pts)
[budget-auditor]       PASS - peak 1076 entities ~ 16.60ms (0.00ms headroom)

DONE - 7 encounter rows, 3 negotiation round(s)
```

THE FINDING THIS PRODUCED FOR KINDLED

The escalating-density promise in the vertical slice cannot be paid for by the current renderer. Floor 3 has to be cut to roughly 55% of its intended body count to hold 60 fps, and even then it lands with zero headroom. This agrees with the project's independent conclusion that the debug-box renderer cannot hold the entity gate — but it was reached from the other direction, by asking what the *encounter* can afford rather than what the renderer costs.

Separately, the readability-auditor reports **CROWDED** on the boss wave: four distinct enemy types against two free palette values, once the lit floor and the retinue have taken theirs. Its verdict is written into the emitted schema, so art direction inherits a stated constraint rather than a surprise.

6 Honest limits

- **The agents reason deterministically.** They read files, interpolate a measured curve, apply design rules and negotiate — they do not call a language model. The assignment permits raw orchestration and this crew uses that permission deliberately, for the reliability reason in §1.
- **The cost curve is single-client and measured on the debug renderer.** When the Niagara sprite path gets a baseline, `BUDGETS.md` changes and every number the crew produces changes with it — which is exactly why `canon-reader` parses that file instead of hardcoding it.
- **Tier multipliers are authored, not derived.** `roster-architect` encodes a designer's ladder; only the baseline it scales is measured.

Kindled — Encounter Design Crew · Aaron Low · 27 July 2026 · Multi-Agent AI for Game Development, Assignment #03. Code `crew/kindled_crew.py`, ReadMe `crew/README.md` (mermaid diagram renders live there), output `crew/out/`. Every transcript in this document is verbatim terminal output from the committed code.